

Make invalid outputs unreachable

Learn unrestricted coordinates; transform them into valid model parameters.

THE QUESTION

How do we keep a network's outputs legal, and how should the step size change during training?

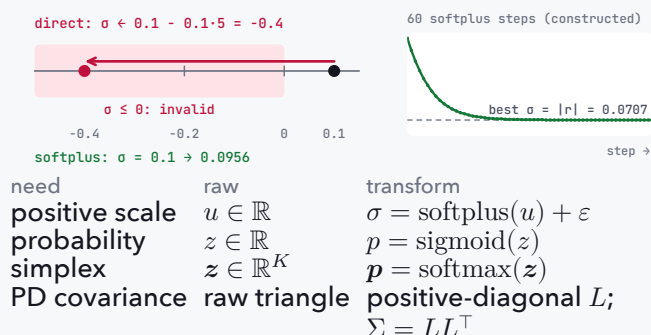
THE INTUITION

Output any real number and map it smoothly into the legal set; gradients flow through the map. Take large steps early and smaller ones late: minibatch noise does not shrink near the minimum.

KEY TAKEAWAY

Constrain by construction, compute in log space, and decay η so late steps bounce less.

1 Constraint map



The optimizer never touches the constrained value directly; the chain rule carries gradients through φ . Above: the slide's $\sigma_0 = 0.1$, gradient 5, $\eta = 0.1$.

2 Positive scales



$\text{softplus}(u) = \log(1 + e^u) > 0$, $\text{softplus}'(u) = \text{sigmoid}(u)$. Use $\sigma = \text{softplus}(u) + \varepsilon$ for a strictly positive numerical floor. Unlike e^u , softplus grows about linearly for large u , so raw steps are gentler. Very negative raw scales still learn slowly (slope 0.018 at $u = -4$). Initialize sensibly.

3 Binary probabilities



$p = \text{sigmoid}(z) = 1/(1 + e^{-z})$. For binary cross-entropy $\ell = \text{softplus}(z) - yz$ and $(\partial \ell)/(\partial z) = p - y$. Train from raw logits with a fused loss; compute p only for reporting.

```
loss = F.binary_cross_entropy_with_logits(
    z, y)          # fused, stable
p = z.sigmoid()    # reporting only
```

Shape guard: `assert y.shape == z.shape; [N]` and `[N,1]` can broadcast into an unintended `[N,N]` loss.

4 Multiclass probabilities

z	e ^z	p
2	7.389	0.665
1	2.718	0.245
0	1.000	0.090

sum e^z = 11.107 · p sums to 1
-log p₁ = 0.408

z = (1000, 999, 998), float32
naive e^z = (inf, inf, inf)
p = inf/inf = (nan, nan, nan)
z - max = (0, -1, -2)
e^{z - max} = (1, 0.368, 0.135)
p = (0.665, 0.245, 0.090)
same p as z = (2, 1, 0)

$p_k = e^{z_k} / \sum_j e^{z_j}$, $p_k > 0$, $\sum_k p_k = 1$. Adding one constant to every logit leaves p unchanged, so subtract $m = \max_j z_j$: $p_k = e^{z_k - m} / \sum_j e^{z_j - m}$. Now every $z_j - m \leq 0$, nothing overflows, and the denominator contains at least one $e^0 = 1$.

5 Stay in log space for training

Stable cross-entropy for target class t : $\ell(z, t) = -(z_t - m) + \log \sum_j e^{z_j - m}$. The same idea fixes softplus: $\max(z, 0) + \log(1 + e^{-|z|})$ gives 1000 at $z = 1000$, where the direct float32 formula gives inf.

```
# logits: [N, K], target: [N] integer classes
loss = F.cross_entropy(logits, target)
log_p = F.log_softmax(logits, dim=-1)
```

Never compute `log(softmax(logits))` as two operations. Stability cannot repair NaN or infinite input logits.

6 Positive-definite covariance



Decode raw outputs into a lower-triangular L with a positive diagonal, then $\Sigma = LL^\top$. For any $a \neq 0$, $a^\top \Sigma a = \|L^\top a\|^2 > 0$, since L is nonsingular. Without that, $\Sigma = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ gives zero variance along $(1, -1)$. In PyTorch distributions pass the factor directly (`scale_tril`).

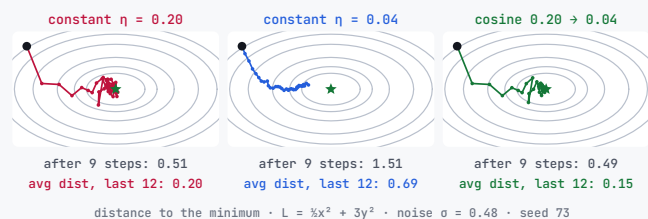
7 Domain audit

✓ write the legal set before choosing the head ✓ keep the raw output unrestricted ✓ use a differentiable map with a useful slope ✓ use fused stable losses on logits ✓ assert shapes, dtypes and finite values

Schedule the learning rate

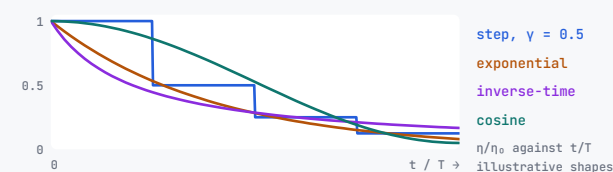
$\theta_{t+1} = \theta_t - \eta_t d_t$: the optimizer builds d_t ; the schedule chooses its scalar scale.

8 Why decay helps



A minibatch gradient is $g_{\mathcal{B}}(\theta) = \nabla \mathcal{L}(\theta) + \varepsilon_{\mathcal{B}}$. Near a minimum the true gradient shrinks but the noise stays: in this run both are about 0.6 (signal 0.63, noise 0.59). The update noise is about $\eta_t \varepsilon_{\mathcal{B}}$, so shrinking η_t reduces the bounce. The cosine run keeps the fast start (0.49 after 9 steps) and bounces least late (0.15). With exact full-batch gradients decay still helps with finer late steps; the noise argument is strongest for SGD. Averages over the last 12 steps are used because a single final point is luck.

9 Four time-based rules



rule
 step $\eta_t = \eta_0 \gamma^{\lfloor t/S \rfloor}$
 exponential $\eta_0 \gamma^t$ (same as $\eta_0 e^{-kt}$, $\gamma = e^{-k}$)
 inverse-time $\eta_0 / (1 + kt)$
 cosine $\eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$

Define the clock: t may count updates, epochs or scheduler calls; the formula is incomplete without it. With a known budget, cosine is a strong simple default; choose T and $\eta_{\min}$ explicitly.

10 Warmup, and letting validation decide



Warmup: $\eta_t = \eta_{\text{peak}} t/W$ for the first W updates, then the decay rule. Use it when starting at the peak rate causes spikes or divergence; it is not mandatory, and it counts toward the update budget. Reduce-on-plateau cuts η by a factor when the validation metric stalls; it must never watch the test set.

11 Which schedule?

situation	simple choice
fixed run length	cosine, or predeclared steps
validation decides	reduce on plateau (val. metric)
fragile startup	warmup, then time-based decay
short exploration	constant, for interpretability

12 Worked: the cosine run above

$\eta_t = 0.04 + 0.16 \cdot (1 + \cos(\pi t/36))/2$ over $T = 36$ updates:

t	0	9	18	27	36
η_t	0.200	0.177	0.120	0.063	0.040

```
sched = optim.lr_scheduler.CosineAnnealingLR(
    opt, T_max=num_epochs, eta_min=1e-4)
```

13 Correct call order

```
for x, y in train_loader:
    opt.zero_grad(set_to_none=True)
    loss = criterion(model(x), y)
    loss.backward()
    opt.step()
    sched.step() # if the clock is one epoch
```

For ReduceLR0nPLateau, evaluate first, then `sched.step(val_loss)`. Log the rate used for each update.

14 Schedule audit

- ✓ the optimizer's base rate was tuned first
- ✓ the clock and total horizon are explicit
- ✓ warmup and decay form one continuous rate trace
- ✓ resuming restores optimizer, scheduler and clock
- ✓ comparisons use the same update budget
- ✓ validation, not test, drives metric-based changes

15 Pause and think

Why not just use the small constant rate? It is still far away: 1.51 after 9 steps and 0.69 on average over the last 12.

Does a schedule replace tuning η_0 ? No. Tune the base rate first; the schedule only shapes it over time.

Do I always need warmup? Only if the first updates at the peak rate spike or diverge.

Choose the simplest schedule that matches the run; add complexity only for a diagnosed failure mode.

Numbers from the Lecture 9 slides' examples recomputed in `briefs/L09-data.py` (noisy SGD: seed 73, 36 steps, one run). The 60-step softplus re-run and the covariance's raw values are constructed from the slides' numbers. Slides and lesson: nipunbatra.github.io/dl-2026.